Known Limitations, Technical Debt & Roadmap

Everything in sections 1–3 was observed in the code. Section 4 is a recommendation for discussion, not a commitment.

1. Architectural limitations

#	Limitation	Consequence	Direction
L1	**Single process.** Connector sessions, the SSH route cache, health results, terminals, live-stat streams, rate limiters and the payment reconcile throttle all live in memory	Cannot run two instances behind a load balancer: connectors would attach to one instance only, rate limits would be split, and the health monitor and auto-deploy poller would run twice	Move shared state to Redis; elect one leader for background jobs; sticky routing for WebSockets
L2	**Background jobs in the web process** (health monitor, auto-deploy poller)	A slow job competes with API requests; a restart pauses monitoring	Separate worker process or job queue
L3	**Deploys run as SSH sessions held by the panel**	A panel restart loses the live log stream; a stuck deploy is released only after 40 minutes	Run deploys detached on the server with a status file; resume on restart
L4	**Schema migrations are imperative code at boot**	No down-migrations, no migration history table	Versioned migrations (for example a `schema_migrations` table)
L5	**The front end is one large file** (`public/app.js`, ~18k lines) with no build step	Harder to navigate, test and share between pages	Split into ES modules per feature (still no bundler needed)
L6	**Ubuntu-centric server scripts**	Other distributions are unsupported	Abstract package-manager calls if demand appears
L7	**Node.js-only build pipeline**	Python, Go, PHP and others need their own `Dockerfile`	Add buildpacks or templates per language
L9	**Setup file is copied by hand**	`aj-pilot-setup.sh` is downloaded in the browser and carried to the machine (USB stick or `scp`); a fresh Ubuntu Server has no browser. Power-on after a power cut is a BIOS setting no script can change. The machine still has to be added in the panel by hand afterwards	A short-lived download link (`curl … | sudo bash`) and a one-time token so the script can register the server itself
L8	**Build-time env needs BuildKit**	The generated Dockerfile uses `RUN --mount=type=secret` when an app has variables; Docker older than 23 without BuildKit cannot build it. A repository's own Dockerfile gets the vars only if it mounts the `appenv` secret itself	Detect the Docker version in the prepare step and explain, or fall back to building without the variables

2. Product gaps

Gap	Notes
No automatic recurring billing	Each cycle is bought on its own; renewals depend on the customer coming back. Razorpay Subscriptions would automate it
No invoices / GST	Payment history exists, but no downloadable invoice
No SMS provider	OTPs are fixed or written to the log until one is integrated
No email notifications for operations	No email for a server going offline, a failed deploy or a trial about to end
Activity log has no filters, retention or export	Only the last 50 entries are shown
No backups of managed databases	Installing MySQL/Postgres/Mongo/Redis/Elasticsearch does not schedule backups (no Elasticsearch snapshot repository is set up either)
Container backups are download-only	The container page's *Backup* archive has no restore button, no schedule and no remote storage; bind-mounted host folders are listed but not copied, and without *pause* a busy database's volume copy may be inconsistent. Restoring is by hand: untar each volume into a fresh volume, then `docker compose up -d`
No rollback to a previous release	Redeploy builds the current branch head; there is no "deploy this earlier SHA"
Terminal permission	Editors can open a shell, which is broader than "never delete". Needs a product decision
`projects` table is unused	Left from an early design; can be dropped

3. Engineering health

Area	Current state	Recommendation
Automated tests	`npm test`: 130 end-to-end tests (public site, auth, platform, roles and tenancy, billing and Razorpay webhook, servers over real SSH incl. terminal, integrations, front-end static checks). See tests/	Add a real git repo + Docker host to cover deploys, domains and installs; add unit tests for `nodeApp` detection
CI	No pipeline in the repository	Run `npm test` on every pull request (needs MySQL + Docker in CI)
Linting / formatting	No config committed	ESLint + Prettier matching the current style
Types	Plain JavaScript	JSDoc types or a gradual move to TypeScript in `src/lib`
Observability	Console logs only	Structured logs (JSON), request ids, metrics (deploy duration, SSH errors, payments)
Dependency hygiene	`npm audit` reports issues	Review and update; pin versions for production builds

Area	Current state	Recommendation
Documentation	Root README (user guide) + this folder	Keep the API Reference generated and checked in CI

4. Suggested roadmap

Now: launch readiness (0–1 month)

1. `OTP_MODE=random` + an SMS provider (e.g. MSG91 or Twilio).
2. Decide the terminal permission (admin-only, or a new `shell` permission).
3. Razorpay webhook configured in production; a reconciliation alert on orders that stay unpaid with an error.
4. Smoke tests for sign-up → payment → plan active, and add server → deploy → domain.
5. Backup automation for the panel database and documented key escrow.

Next: retention and revenue (1–3 months)

1. Recurring billing (Razorpay Subscriptions) and GST invoices (PDF) in payment history.
2. Notifications by email and webhook: deploy failed, server offline, plan ending in 5 days.
3. Rollback to a previous successful deployment.
4. Activity log filters, export and retention.
5. Scheduled backups for installed databases, to S3-compatible storage.

Later: scale and breadth (3–6 months)

1. Shared state in Redis and a separate worker → multiple panel instances.
2. Build templates for Python, Go and PHP; Docker Compose stacks from a repository.
3. TOTP two-factor and OIDC single sign-on for teams.
4. Public API with per-organisation API keys, for CI integrations.
5. Second payment gateway (Stripe) for markets outside India.

5. Decision log (to maintain)

Record significant decisions here as they are made:

Date	Decision	Reason	Alternatives considered
—	SSH only, no agent on managed servers	Nothing to install or keep updated; works with any VPS	Agent-based (more real-time, more maintenance)
—	No front-end build step	Simplicity; static files served directly	React/Vite SPA
—	Paid plan activates only after server-side verification	Browser responses can be forged	Trust the checkout callback
—	xterm.js served from `node_modules`, not a CDN	Keeps the CSP to self + Razorpay	CDN with CSP exception